안녕하세요.
Jquery 강좌중인 Falcon 입니다.

오늘은 여러분들에게 드디어 이벤트라는 것에 대해서 알려드리도록 하겠습니다. 이벤트는 쉽게 말하면 어떠한 사건을 의미합니다. 웹 브라우저에서 사건이라는 것은 마우스, 키보드, 스크롤 등에 대한 이벤트를 의미 합니다.

```
7   <body>
8     <input type="button" onclick="alert('Hello World');" value="alert()" />
9   </body>
```

index.html

이벤트를 등록하는 방법으로는 3가지 방법이 있습니다. 먼저 제가 설명을 하면서 가장 많이 보여드렸을 법한 inline 방식 입니다. 여기서 onclick는 클릭이벤트를 발생시키겠다는 의미 그리고 다음으로 보이는 것은 어디선가 많이 본듯한 느낌이 드실겁니다. 이유는 그동안 사용해오던 javascript의 문법이니까요.

이 부분을 실행하면 이렇게 버튼이 하나 생기게 됩니다. 이 버튼을 클릭하면 클릭이벤트가 발생하게 됩니다.

그러면 Hello World라는 문자열이 출력되는 것을 볼 수 있습니다. 이 방법외에도 사용했던 것은 함수를 이용하는 방법이 있었습니다. 그 부분을 또 다시 복습을 해보도록 합시다.

```
8   <body>
9     <input type="button" onclick="Alert()" value="alert()" />
10  </body>
```

index.html

inline 방식은 단점이 하나가 존재합니다. 그는 해야할 일이 많아지게 되면 소스코드가 많이 더러워진다는 단점이 있습니다. 그런데 그 부분은 함수를 이용하면 해결이 가능합니다. 위처럼 Alert() 라는 함수를 호출했습니다.

```
1  function Alert() {
2    alert("Hello World");
3  }
```

master.js

master.js파일을 생성시에는 모듈화를 해줘야한다는 사실을 잊지마시고 작업하시기 바랍니다. 사실 모듈화를 하지 않아도 이 방법을 구현할 수 있습니다. 저는 강좌의 쓸 때 편하게 설명하기 위해서 모듈화를 사용합니다. 위의 실행문의 결과값은 위에서 보여드린 것과 동일 합니다. 그런데 이 inline 방식은 추천드리지 않습니다. 제가 강좌를 하면서도 가장 많이 사용했던 방법이기는 하지만 HTML과 javascript가 혼재되어 있는 형태이기 때문에 바람직한 방법이라고는 할 수 없다고 합니다.

```
8     <body>
9       <input type="button" id="Alert" value="alert()" />
10    </body>
```

index.html

이번에 보여드릴 방식은 property 방식입니다. property방식은 속성을 이용해서 등록을 하는 방식입니다.

```
1   window.onload = function() {
2     var t = document.getElementById('Alert');
3     t.onclick = function(event) {
4       alert("Hello World");
5     };
6   };
```

master.js

여기서 여러분들이 궁금해하실 수 있는 사항이 있습니다. 바로 window.onload가 위에는 들어가 있지 않은데 여기서는 사용되고 있습니다. 그 이유는 모듈화를 하게되면 2행에 위치한 id값이 Alert라는 것을 찾을 수 없기 때문입니다. 하지만 function Alert() 함수 같은 경우는 그 함수가 클릭되었을 때 호출을 하기 때문에 찾을 수 있습니다.

위에서 설명드릴 부분은 딱히 없습니다. t.onclick는 우리가 inline 방식에서 사용했던 속성 onclick 속성을 넣어준 것 입니다. 단지 html의 문서에서는 onclick라는 속성이 보이지 않습니다.

```
▼<body>
   <input type="button" id="Alert" value="alert()">
 </body>
```

개발자 도구로 확인을 해보시면 input에는 onclick라는 속성이 보이지 않는 것을 볼 수 있으실겁니다.
이 부분에 대해서는 inline에서 단점으로 지적되었던 HTML문서와 Javascript가 분리된 것을 볼 수 있습니다.

```
1   window.onload = function() {
2     var t = document.getElementById('Alert');
3     t.onclick = function(event) {
4       alert(event.target.value);
5     };
6   };
```

master.js

이번시간에서 지금까지 여러분들에게 설명드리지 않았던 부분들은 한번 설명해보려고 합니다. 여러분들이 추후 웹을 직접 개발을 하게 되면 많은 사람들이 자신이 가장 많이 사용했던 브라우저를 통해서 테스트를 하고 확인을

하게 됩니다. 그런데 사용자의 웹 브라우저는 여러분들이 사용하는 웹 브라우저와 다를 수 있습니다. master.js의 파일을 이렇게 수정을 하고 클릭을 해봅시다.

위에서 보이는 event.target은 쉽게 얘기해서 여러분들이 호출한 엘리먼트를 뜻합니다. 그 안의 value값을 가져오라고 했으니 당연히 value값 alert() text를 가져오게 되는데 이를 ie8 이하에서 실행을 하면 실행되지 않습니다.

```
1 window.onload = function() {
2   var t = document.getElementById('Alert');
3   t.onclick = function(event) {
4     alert(event.target.value);
  ⊗ 정의되지 않음 또는 null 참조인 'target' 속성을 가져올 수 없습니다.
```

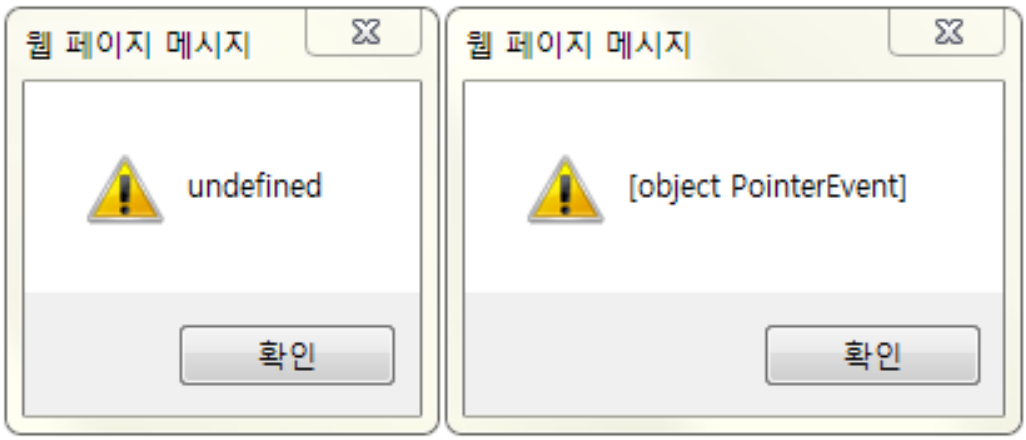
위에서는 단순히 target의 속성을 가져올 수 없습니다라는 오류가 나오는데 실제로는 핸들러의 인자값 event 자체도 undefined가 뜨기때문에 가져올 수 없습니다. 이러한 것들이 크로스 브라우징의 일부 입니다.

크로스 브라우징이란 html, css, javascript 작성시 W3C의 web strandard(웹 규격)에 맞는 코딩을 하면서 어느 브라우저나 기기에서 사이트가 제대로 보여지고 작동되도록 하는 기법을 말한다. 즉, 위의 javascript 코드는 ie8 이하 버전에서는 사용할 수 없는 기법이라는 것이다. 위에서 event, target에 대한 오류가 발생한다.

```
1  window.onload = function() {
2    var t = document.getElementById('Alert');
3    t.onclick = function(event) {
4      event = event || window.event;
5      var target = event.target || event.srcElement;
6      alert(target.value);
7    };
8  };
```

master.js

ie8이하에서 event는 핸들러의 인자가 아니라 전역객체의 property로 제공됩니다. 그렇기 때문에 ie8에서는 event를 호출하게 되면 undefined라는 아무런 값이 없다라고 출력 됩니다. 여기서 핸들러 인자는 function 옆에 있는 인자 값을 의미하고 이 함수를 호출하면 생기는 현상(?)을 핸들러라고 생각하시면 됩니다.



위를 보시면 undefined는 ie8이하에서 출력을 한 모습 그리고 Object PointerEvent는 ie9이상에서 출력한 결과입니다. 이처럼 ie8이하에서는 핸들러 옆의 인자값을 읽어들일 수 없기 때문에 이를 정상적으로 돌아가게끔 만들어줘야 합니다. 그부분이 위의 코드에서 4행에 위치하고 있습니다. 4행의 코드를 보면 이렇습니다.

```
event = event || window.event
```

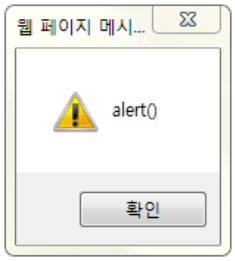
여기까지오신분들은 || 이부분에 의미를 모를리 없을것 입니다. 그런데 이걸보면 조금 의아해하실 수 있습니다.

```
if(num == 1 || num == 2)
```

보통 이러한 조건형식에서 많이 보셨을텐데 위는 그러한 조건이 없는 것고 단지 event || window.event라고 되어있습니다. 이는 쉽게 말하면 event에 값이 없다면 window.event를 사용해라라고 생각하시면 됩니다. 위에서 결과로 보여드리듯 event에는 ie8에서는 undefined라는 값이기 때문에 값이 없습니다. 그렇기 때문에 이 경우 위에서 설명드린 전역객체의 window.event를 불러오게 됩니다.

```
target = event.target || event.srcElement
```

ie8에서는 event.target이 event.srcElement가 됩니다. 즉, 여기에서도 event.target이 없다면 event.srcElement를 사용해라라고 생각하시면 됩니다.



그리고나서 ie8에서 위의 코드를 실행을 해보면 이처럼 정상적으로 alert()가 출력되는 것을 볼 수 있습니다. 마지막으로 여러분들에게 알려드릴 방식은 addEventListener()을 통해서 등록하는 방식을 가르쳐드리도록 하겠습니다. 개인적으로 저는 프로젝트를 할 때 inline방식을 많이썻는데 이게 가장 권장하는 방식이라고 합니다.

```
1   window.onload = function() {
2     var t = document.getElementById('Alert');
3     t.addEventListener('click', function(event) {
4       alert("Hello World");
5     });
6   };
```

master.js

위에서 보이듯이 사용법은 이렇습니다. t.addEventListener()은 메소드 입니다. 첫번째 인자값으로는 이벤트를 선택합니다. 저는 click 클릭에 대한 이벤트를 지정했고, 두번째 인자값은 보시다시피 property에서 보던 실행문입니다. 그런데 여기서도 ie8이하 버전에 대한 문제점이 생기게 됩니다.

```
1 window.onload = function() {
2   var t = document.getElementById('Alert');
3   t.addEventListener('click', function(event) {
4     alert("Hello World");
5   });
  ⊗ 개체가 'addEventListener' 속성이나 메서드를 지원하지 않습니다.
```

ie8이하버전에서는 addEventListener이라는 속성이나 메서드를 지원하지 않는다고 합니다.

```
1   window.onload = function() {
2     var t = document.getElementById('Alert');
3     if(t.addEventListener) {
4       t.addEventListener('click', function(event) {
5         alert("Hello World");
6       });
7     } else if(t.attachEvent) {
8       t.attachEvent('onclick', function(event) {
9         alert(event);
10      });
11    }
12  };
```

master.js

ie8에서 이 기능을 사용할 때에는 attachEvent라는 메소드를 사용해야하기 때문입니다. 그렇기 때문에 ie8이하 버전에서 사용하려면 위처럼 지정을 해줘야 가능합니다. 9행을 보면 event를 그대로 사용하고 있지만 property 에서와는 달리 이 기능을 사용할 때에는 핸들러의 인자로 event가 지원이 되니 이점도 알아두시기 바랍니다.

```
1   window.onload = function() {
2     var t = document.getElementById('Alert');
3     t.addEventListener('click', function(event) {
4       alert("Hello World");
5     });
6     t.addEventListener('click', function(event) {
7       alert(event.target.value);
8     });
9   };
```

master.js

이 방법의 중요한 장점은 하나의 이벤트 대상에 복수의 동일 이벤트 타입 리스너를 등록할 수 있다는 점이 장점입니다. 이벤트가 Click만 있는 것이 아닙니다. 예를들어서 마우스가 위로 올라왔을 때 내려갈 때 등을 생각한다면 이는 엄청난 장점입니다. HTML코드와 분리해서 작업을 하면서 처리를 할 수 있으니까요.

```
8   <body>
9     <input type="button" id="Alert1" value="alert1()" />
10    <input type="button" id="Alert2" value="alert2()" />
11  </body>
```

index.html

이부분에 대해서는 설명할게 없습니다. 단지 버튼을 하나더 추가했을 뿐이고 id 값이 1과2로 나뉘어져 있습니다.

```
1   window.onload = function() {
2     var t1 = document.getElementById('Alert1');
3     var t2 = document.getElementById('Alert2');
4     function btn_listener(event) {
5       switch(event.target.id) {
6         case 'Alert1' :
7           alert(1);
8           break;
9         case 'Alert2' :
10          alert(2);
11          break;
12      }
13    }
14    t1.addEventListener('click', btn_listener);
15    t2.addEventListener('click', btn_listener);
16  };
```

master.js

2행과 3행에서 엘리먼트들의 id값을 불러오고 14행과 15행에서 호출을 해주고 있습니다. 자 여기서 두번째 인자값은 function(event)라고 알려드렸는데 함수도 마찬가지이기 때문에 함수명을 호출해도 상관이 없습니다. 그렇기 때문에 4행 부분이 호출이되고, 5행에서 swidth문 event.target.id의 값에 따라서 출력이되는 것을 볼수 있으실 겁니다. 이 부분은 사실 언어에 대한 기본적인 이해만 하고 계셔도 충분히 분석할 수 있는 내용입니다. 이렇게해서 여러분들에게 event의 등록방법에 대한 모든 설명을 마치면서 저는 이만 마무리를 하도록 하겠습니다.

이것으로 [Jquery 강좌] 제 30강 이벤트에 대해서 알아보자 편을
마치도록 하겠습니다.